

Lecture 4: CPU Scheduling & Process Management

1. Course Roadmap & Core Objectives

CPU scheduling is the basis of multiprogrammed operating systems. By switching the CPU among processes, the OS makes the computer more productive.

Key Learning Objectives for this Chapter:

- Understand the alternating CPU-I/O burst cycle and process behaviors.
- Differentiate preemptive from non-preemptive CPU dispatching.
- Derive and apply quantitative formulas for Turnaround Time (TAT) and Waiting Time (WT).
- Analyze classical algorithms: FCFS, SJF, SRTF, Round Robin, and Priority Scheduling.
- Master advanced scheduling architectures: Multilevel Feedback Queues (MLFQ) and Dynamic Aging.

Exam Relevance: Scheduling algorithms and Gantt chart calculations appear with high frequency in midterms.

Lecture 4: CPU Scheduling Principles

2. The CPU-I/O Burst Cycle

Process execution consists of an alternating cycle of CPU execution and I/O wait. Processes alternate between these two phases. Execution begins with a CPU burst, followed by an I/O burst, followed by another CPU burst, eventually concluding with a final CPU burst terminating execution.

Burst Distribution Properties:

The distribution of CPU bursts across diverse workloads is characterized as hyperexponential:

- **High frequency of short bursts:** Characterizes I/O-bound processes (interactive applications, text editors, GUI event loops).
- **Small frequency of long bursts:** Characterizes CPU-bound processes (matrix multiplications, scientific simulations, video encoding).

Definition — I/O-bound process: A process that spends more of its time performing input/output operations than executing processor computations.

Lecture 4: CPU Scheduling Principles

3. CPU Scheduler & Dispatcher Latency

Whenever the CPU becomes idle, the operating system must select one of the processes in the ready queue to be allocated the CPU. This selection process is performed by the short-term scheduler (or CPU scheduler).

Role of the Dispatcher:

The dispatcher is the component that actually gives control of the CPU to the process selected by the short-term scheduler. Its responsibilities include:

- Switching context from the prior process to the new process.
- Switching the CPU hardware into user mode.
- Jumping to the proper instruction location in the user program to restart execution.

Dispatch Latency Formula & Tradeoff:

Dispatch Latency = Context Save Time + Context Restore Time + State Transition Time

Dispatch latency is pure system overhead during which no useful application work occurs. Schedulers must minimize dispatch latency.

Lecture 4: CPU Scheduling Principles

4. Preemptive vs. Non-Preemptive Scheduling

CPU-scheduling decisions take place under four specific operating conditions:

- **Condition 1:** When a process switches from running state to waiting state (e.g. for I/O completion).
- **Condition 2:** When a process switches from running state to ready state (e.g. timer interrupt).
- **Condition 3:** When a process switches from waiting state to ready state (e.g. completion of I/O).
- **Condition 4:** When a process terminates execution.

Classification Rule:

- **Non-Preemptive (Cooperative):** Scheduling takes place ONLY under Conditions 1 and 4. Once a process gets the CPU, it holds it until voluntary release.
- **Preemptive:** Scheduling can occur under Conditions 2 and 3. The kernel can preempt a running process to allocate CPU to higher-priority or newly ready tasks.

Crucial Exam Pitfall: Preemption requires race-condition guards when kernel data structures are updated concurrently.

Lecture 4: Scheduling Optimization Criteria

5. Optimization Criteria & Goals

Different algorithms favor different criteria. In designing schedulers, operating systems seek to optimize conflicting performance targets:

- **CPU Utilization:** Percentage of time CPU is busy executing user/system code. (Target: Maximize, 40% to 90%).
- **Throughput:** Number of complete processes executed per unit time. (Target: Maximize).
- **Turnaround Time:** Interval from submission to completion. (Target: Minimize).
- **Waiting Time:** Cumulative duration process spent waiting in ready queue. (Target: Minimize).
- **Response Time:** Elapsed duration from request arrival until the first response output. (Target: Minimize).

System Focus: Batch systems maximize throughput and minimize turnaround time; interactive time-sharing systems prioritize minimizing response time.

Lecture 4: Performance Metric Formulas

6. Quantitative Formulas for Exam Problems

Students must memorize and correctly apply these core mathematical equations during Gantt chart analysis:

Mathematical Formula 1 — Turnaround Time (TAT):

$TAT = \text{Completion Time } (T_{\text{exit}}) - \text{Arrival Time } (T_{\text{arrival}})$

Note: If a process arrives at $t > 0$, you must subtract arrival time.

Mathematical Formula 2 — Waiting Time (WT):

$WT = \text{Turnaround Time } (TAT) - \text{CPU Burst Time } (BT)$

Waiting time is the pure idle delay spent inside the ready queue awaiting processor scheduling.

Mathematical Formula 3 — Average Waiting Time:

$\text{Average WT} = (\sum_{i=1}^n WT_i) / n$

Common Mistake in Exams: Confusing Response Time with Waiting Time. In non-preemptive systems they are equal, but in preemptive systems they diverge drastically.

Lecture 4: Classical Scheduling Algorithms

7. First-Come, First-Served (FCFS) Scheduling

The process that requests the CPU first is allocated the CPU first using a standard FIFO queue. When a process enters the ready queue, its PCB is linked onto the tail of the queue.

Characteristics:

- FCFS is strictly non-preemptive.
- Implementation is straightforward using a FIFO queue.
- However, average waiting time under FCFS is often quite long and highly variable depending on process arrival order.

Numerical Example:

Consider three processes P1 (Burst=24), P2 (Burst=3), P3 (Burst=3) arriving at t=0 in order P1, P2, P3.
Completion: P1=24, P2=27, P3=30. Waiting times: P1=0, P2=24, P3=27.
Average Waiting Time = $(0 + 24 + 27) / 3 = 17.0$ ms.

Lecture 4: Classical Scheduling Algorithms

8. The Convoy Effect in FCFS

If the arrival order was P2, P3, P1 instead, average waiting time becomes $(0 + 3 + 6) / 3 = 3.0$ ms. This 5.6x difference illustrates the major vulnerability of FCFS.

Definition — The Convoy Effect:

The convoy effect is a phenomenon where numerous short I/O-bound processes become blocked in the ready queue behind a single long CPU-intensive process.

Impact on System Utilization:

- While the long CPU process executes, all I/O devices sit completely idle.
- When the CPU process finally yields for I/O, all short processes rush through the CPU quickly and then crowd the I/O queues simultaneously, leaving the CPU idle.
- Result: Extremely poor device utilization and degraded interactive system responsiveness.

Lecture 4: Classical Scheduling Algorithms

9. Shortest-Job-First (SJF) Scheduling

Shortest-Job-First associates each process with the length of its next CPU burst. When the CPU becomes available, it is assigned to the process having the smallest next CPU burst duration.

The Optimality Theorem:

Theorem: SJF is provably optimal because it yields the minimum average waiting time for any given set of stationary processes.

Proof Intuition: Moving a shorter burst ahead of a long burst reduces the waiting time of the short process significantly more than it increases the waiting time of the long process, decreasing overall average waiting time.

Major Practical Obstacle:

The real difficulty with SJF is knowing the length of the next CPU burst in advance. In general-purpose operating systems, future burst length cannot be known ahead of execution.

Lecture 4: Classical Scheduling Algorithms

10. SRTF & Exponential Smoothing Prediction

Preemptive SJF — Shortest-Remaining-Time-First (SRTF):

When a new process arrives with a CPU burst shorter than the remaining execution time of the currently executing process, the current process is immediately preempted.

Exponential Smoothing Prediction Formula:

Since future burst lengths are unknown, schedulers predict them using an exponential weighted average of observed historical bursts:

$$\tau_{n+1} = \alpha * t_n + (1 - \alpha) * \tau_n$$

Where:

- t_n = actual observed length of the n th CPU burst.
- τ_n = prior predicted value.
- α = weighting parameter ($0 \leq \alpha \leq 1$).
 - If $\alpha = 0$, recent history ignored (constant prediction).
 - If $\alpha = 1$, only most recent burst matters (ignoring overall trend).
 - Typical value: $\alpha = 0.5$.

Lecture 4: Classical Scheduling Algorithms

11. Round Robin (RR) Scheduling

Round Robin scheduling is designed specifically for time-sharing systems. The ready queue is managed as a circular FIFO queue. The CPU scheduler allocates the CPU to each process for a fixed time slice termed a **time quantum (q)** (typically 10 to 100 milliseconds).

Execution Mechanics:

- If process burst $\leq q$: Process releases CPU voluntarily; scheduler advances to next ready process.
- If process burst $> q$: Operating system timer fires an interrupt; context switch occurs; process is preempted and returned to the tail of the ready queue.

Performance Dependency on Quantum Size (q):

- If $q \rightarrow \infty$: Round Robin degenerates into First-Come, First-Served (FCFS).
- If $q \rightarrow 0$: Context switch thrashing dominates processor execution cycles.

Lecture 4: Classical Scheduling Algorithms

12. Context Switch Overhead & The 80% Rule

A context switch requires saving process registers into the Process Control Block (PCB), reloading the registers of the next process, and flushing memory caches and TLBs. This is non-productive overhead.

Context Switch Overhead Analysis:

Suppose a context switch takes 10 microseconds. If the time quantum is 100 microseconds, then 10% of CPU time is consumed solely by context switching overhead! Schedulers must keep quantum duration substantially larger than context switch latency.

The 80% Rule of Thumb for Examination Problems:

Rule: 80% of CPU bursts should be shorter than the time quantum (q).

This heuristic ensures that the majority of interactive bursts complete without preemption, while CPU-bound processes are gracefully shared.

Common Mistake: Claiming smaller quantum always improves average turnaround time. In fact, if quantum is too small, turnaround time increases due to context switch penalties.

Lecture 4: Priority Scheduling

13. Priority Scheduling Principles & Starvation

In Priority Scheduling, an integer priority number is associated with each process. The CPU is allocated to the process with the highest priority (by standard Unix convention, smallest integer indicates highest priority). Equal-priority processes are scheduled in FCFS order.

Priority Assignment Types:

- **Internal Priorities:** Derived from OS metrics (memory limits, file counts, ratio of average I/O burst to average CPU burst).
- **External Priorities:** Assigned by external policy criteria (financial department, user seniority, process importance).

The Major Failure Mode — Starvation (Indefinite Blocking):

A major problem with priority scheduling is starvation: a runnable process with low priority may wait indefinitely for the CPU if a continuous stream of higher-priority processes enters the system.

Historical Fact: In 1973, when MIT phased out an IBM 7094, they found a low-priority process submitted in 1967 that had not yet run!

Lecture 4: Priority Scheduling & Starvation Resolution

14. Starvation Resolution via Dynamic Aging

Operating systems resolve indefinite blocking through a dynamic priority adjustment mechanism known as **Aging**.

Definition — Dynamic Aging:

Aging is a technique of gradually increasing the priority of processes that wait in the system for long periods of time.

Aging Algorithmic Implementation:

- For example, if priorities range from 127 (lowest) to 0 (highest), the kernel scheduler can increase the waiting process priority by 1 every 15 minutes.
- Over time, even a process initiated with lowest priority 127 will gradually advance towards priority 0.
- Within at most 32 hours, the oldest waiting process is guaranteed to possess priority 0 and gain processor execution.

Exam Differentiator: Distinguish Starvation from Deadlock. Starved processes are runnable and will run if priorities change; deadlocked processes are permanently blocked waiting on unavailable resources held by each other.

Lecture 4: Advanced Scheduling Architectures

15. Multilevel Feedback Queue (MLFQ) Scheduling

In simple Multilevel Queue scheduling, processes are permanently assigned to one queue based on type (Foreground interactive vs Background batch). This lacks adaptability. The **Multilevel Feedback Queue (MLFQ)** solves this by allowing processes to dynamically migrate between queues.

Key Parameters Defining an MLFQ Scheduler:

- **1. Number of queues:** Typically 3 queues (Queue 0, Queue 1, Queue 2).
- **2. Scheduling algorithm for each queue:** E.g., Queue 0 (RR $q=8\text{ms}$), Queue 1 (RR $q=16\text{ms}$), Queue 2 (FCFS).
- **3. Demotion algorithm:** A process that consumes too much CPU time is demoted to a lower-priority queue.
- **4. Promotion algorithm (Aging):** A process that waits too long in a lower queue is boosted to a higher queue to prevent starvation.
- **5. Initial queue placement:** When a process enters the system, it enters Queue 0 (highest priority) with a short quantum.

Importance: MLFQ is the foundation for modern Linux (O(1) and CFS) and Windows NT scheduler architectures.

Lecture 4: Chapter Review & Exam Cheatsheet

16. Summary Comparison & Final Exam Cheatsheet

Summary Comparison Table for Quick Revision:

- **FCFS:** Non-preemptive | $O(1)$ complexity | Major Pitfall: Convoy Effect | Optimal: Never.
- **SJF:** Non-preemptive | Minimizes average WT (Optimal) | Major Pitfall: Requires burst prediction.
- **SRTF:** Preemptive SJF | Provably optimal average WT | High preemption context switch overhead.
- **Round Robin:** Preemptive | Best for interactive response time | Requires 80% rule for quantum q .
- **Priority:** Can be preemptive or non-preemptive | Major Pitfall: Starvation | Solution: Dynamic Aging.
- **MLFQ:** Highly adaptive | Separates I/O-bound from CPU-bound | Prevents starvation via priority promotion.

Core Formula Cheatsheet:

1. $TAT = \text{Exit Time} - \text{Arrival Time}$
2. $WT = \text{Turnaround Time} - \text{Burst Time}$
3. $\tau_{n+1} = \alpha * t_n + (1 - \alpha) * \tau_n$